

**PUNJAB UNIVERSITY COLLEGE OF INFORMATION
TECHNOLOGY(PUCIT)**

Project Title:

Employee Payroll Management System

Submitted To:

Dr.Asif Sohail

Submitted By:

Name	Roll Number
Fiza Majeed	BITF24M007
Mahnoor Ghani	BITF24M013
Abriza Amin	BITF24M017

Employee Payroll Management System

INTRODUCTION OF THE EMPLOYEE PAYROLL SYSTEM	3
ERD OF THE SYSTEM	4
BOTTOM-UP APPROACH	5
TOP-DOWN APPROACH	5
DESCRIPTION OF RELATIONS	8
DESCRIPTION OF RELATIONS(In table form)	13
TABLES CREATION USING SQL	17
VIEWS CREATION USING SQL	29
SELECT STATEMENTS FOR COMMON REPORTS	31
FUNCTIONS	34
STORED PROCEDURES	35
DATABASE TRIGGERS	36
SEQUENCES	37
INDEXES	38

INTRODUCTION OF THE EMPLOYEE PAYROLL SYSTEM

The Employee Payroll Management System is a comprehensive database-driven solution designed to streamline and automate employee administration, payroll processing, and organizational record management within an organization. The system centrally manages employee-related information, including unique employee identification (Employee_ID), personal details such as names, CNIC, contact information, hire dates, and email addresses, ensuring accurate and consistent data storage.

The organizational structure is maintained through the integration of the EMPLOYEE, DEPARTMENT, and JOB_POSITION tables. Each employee is associated with a specific department (Department_ID) and job position (Job_Position_ID), enabling effective departmental management and role-based classification. The hierarchical relationship within the organization is supported through the Manager_ID attribute, which establishes reporting lines by referencing employees who serve as managers.

The system includes an ATTENDANCE module that records daily employee attendance, capturing check-in and check-out times along with attendance status. Additionally, the LEAVE_RECORDS table manages employee leave requests by tracking leave types, durations, and approval statuses. Overtime work is handled through the OVERTIME_RECORDS table, which stores information about extra working hours and applicable overtime rates, contributing directly to payroll calculations.

Employee compensation and benefits are comprehensively managed through the PAYROLL, BENEFITS, and DEDUCTIONS modules. The PAYROLL table maintains monthly salary records, including basic salary, total benefits, total deductions, and net salary. To ensure data normalization and flexibility, benefits are modeled using a supertype-subtype structure, where the BENEFITS table serves as a supertype and is further specialized into INSURANCE and ALLOWANCE subtypes. This structure allows the system to handle multiple benefit categories efficiently while avoiding redundancy. Employee deductions and tax obligations are managed through the DEDUCTIONS and TAX_RECORDS tables, providing transparency in salary adjustments and statutory compliance.

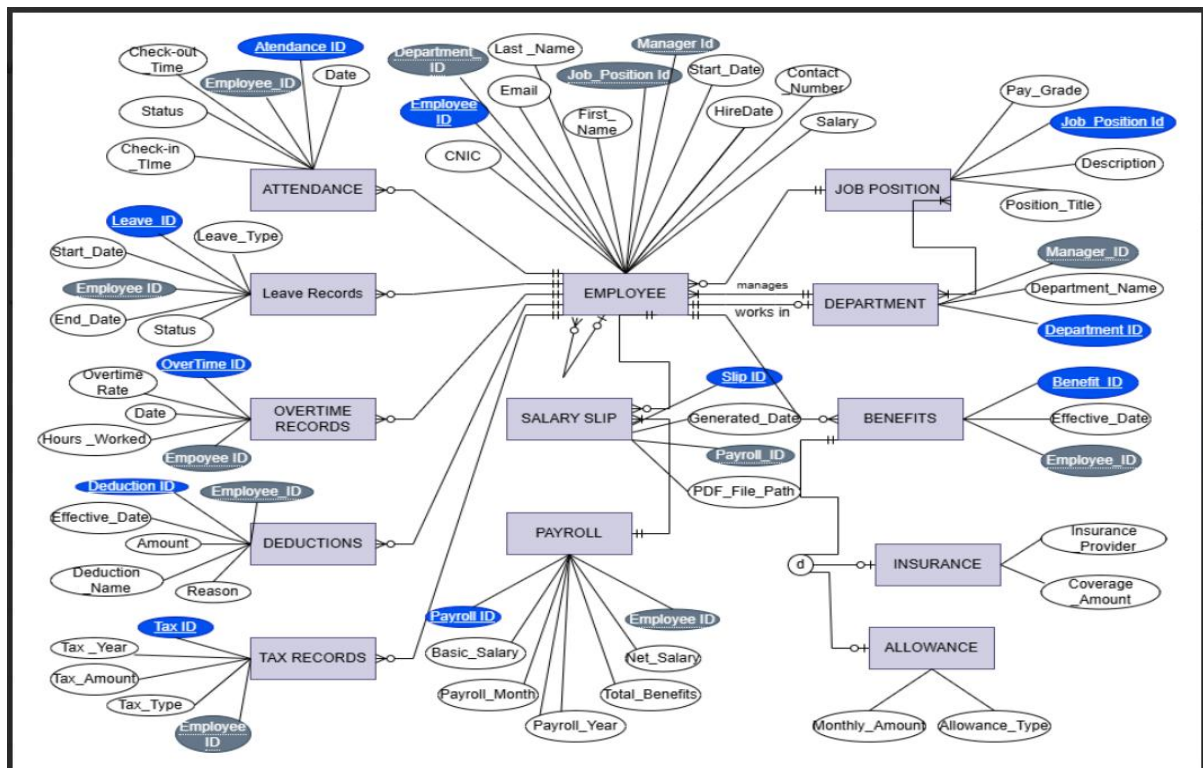
The system also supports payroll documentation through the SALARY_SLIP table, which stores generated salary slips and their corresponding file paths. Advanced database features such as views, functions, stored procedures, triggers, sequences, and indexes are implemented to enhance data integrity, automate payroll generation, enforce business rules, and optimize query performance.

Overall, this system integrates all essential components required for effective employee management, attendance tracking, payroll processing, and financial reporting. By ensuring proper normalization and strong relational integrity, the Payroll System provides a scalable framework for improving administrative efficiency and accuracy within an organization.

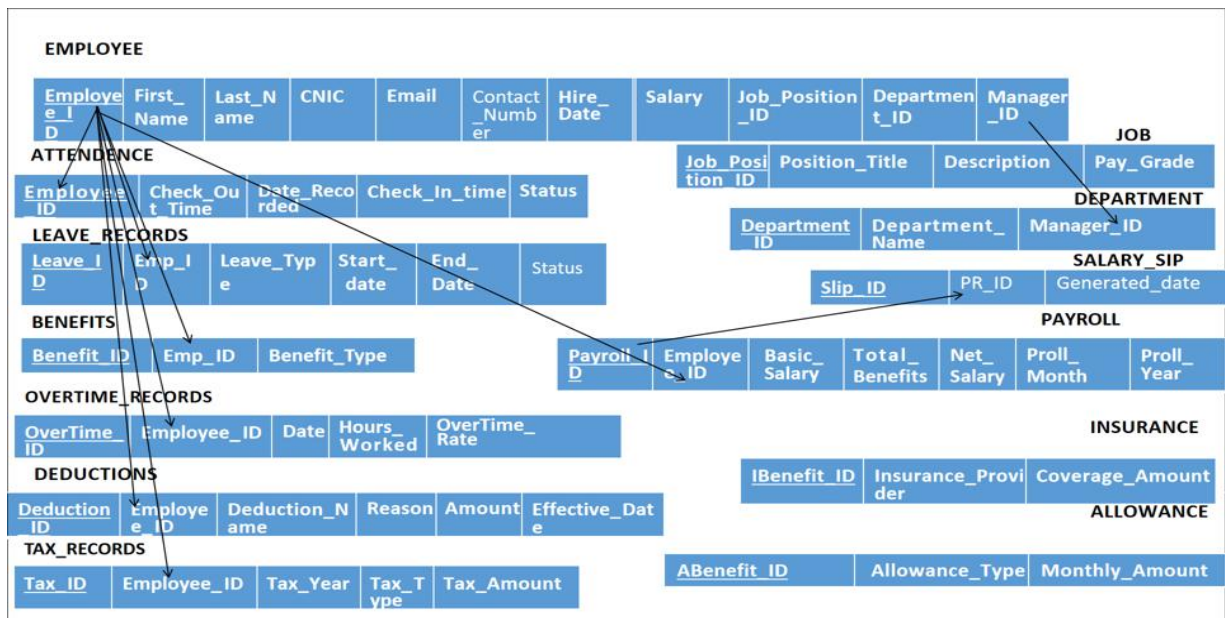
ERD OF THE SYSTEM

Entity	Relationship	Connectivity	Entity
EMPLOYEE	Works in	1:M	DEPARTMENT
EMPLOYEE	Manages	1:1	DEPARTMENT
EMPLOYEE	Holds	1:1	JOB POSITION
EMPLOYEE	Reports to	1:1	EMPLOYEE
EMPLOYEE	Has	1:M	ATTENDANCE
EMPLOYEE	Has	1:M	LEAVE RECORDS
EMPLOYEE	Has	1:M	OVERTIME RECORDS
EMPLOYEE	Has	1:M	DEDUCTIONS
EMPLOYEE	Has	1:1	SALARY SLIP
EMPLOYEE	Has	1:M	TAX RECORDS
EMPLOYEE	Has	1:M	BENEFITS
DEPARTMENT	Has	1:M	JOB POSITION
BENEFITS	Specialized into	1:1	INSURANCE
BENEFITS	Specialized into	1:1	ALLOWANCE
PAYROLL	Generates	1:1	SALARY SLIP

Employee Payroll Management System



BOTTOM-UP APPROACH



TOP-DOWN APPROACH

First Normal Form (1NF)

A relation is in First Normal Form (1NF)

Employee Payroll Management System

- if all attributes contain atomic values
- there are no repeating groups or multi-valued attributes.

Relations after applying 1NF:

- EMPLOYEE(EmployeeID, FirstName, LastName, CNIC, HireDate, Email, DeptID, JobPositionID, ManagerID, EmploymentType, Contact_Number)
- ATTENDANCE(Attendance_ID, EmployeeID, Date, CheckInTime, CheckOutTime, Status)
- LEAVE_RECORDS(LeaveID , EmployeeID, LeaveType, StartDate, EndDate, Status)
- OVERTIME_RECORDS(OvertimeID , EmployeeID, Date, HoursWorked, OvertimeRate)
- DEDUCTIONS(DeductionID, EmployeeID, DeductionName, Amount, EffectiveDate, Reason)
- TAX_RECORDS(TaxID , EmployeeID, TaxYear, TaxAmount, TaxType)

Second Normal Form (2NF)

A relation is in Second Normal Form (2NF)

- if it is already in 1NF
- has no partial dependency.
- Partial dependency can only occur when a composite primary key exists.

The following relations have single-attribute primary keys and are therefore already in 2NF. They were separated to avoid redundancy, not because of partial dependency.

- DEPARTMENT(DepartmentID , DepartmentName, ManagerID)
- JOB_POSITION(JobPositionID, PositionTitle, PayGrade, JobDescription, BaseSalary)
- BENEFITS(BenefitID, EmployeeID, BenefitType, EffectiveDate)
- PAYROLL(PayrollID, EmployeeID, PayrollMonth, PayrollYear, BasicSalary)
- SALARY_SLIP(SlipID , PayrollID, GeneratedDate, PDFFilePath)

Third Normal Form (3NF)

A relation is in Third Normal Form (3NF) if it is in 2NF and contains no transitive dependencies.

Derived attributes such as TotalDeductions, TotalBenefits, and NetSalary were removed from the PAYROLL relation to prevent transitive dependency.

- PAYROLL(PayrollID, EmployeeID, PayrollMonth, PayrollYear, BasicSalary)

Employee Payroll Management System

➤ PAYROLL_DEDUCTIONS(PayrollID, DeductionID, Amount)

➤ PAYROLL_BENEFITS(PayrollID, BenefitID, Amount)

Boyce-Codd Normal Form (BCNF)

A relation is in BCNF if for every functional dependency $X \rightarrow Y$, X is a candidate key.

Since CNIC uniquely identifies an employee, the dependency $CNIC \rightarrow EmployeeID$ violates BCNF.

➤ EMPLOYEE_ID_MAPPING(CNIC, EmployeeID)

➤ EMPLOYEE(EmployeeID, FirstName, LastName, HireDate, Email, DeptID, JobPositionID, ManagerID, EmploymentType)

Fourth Normal Form (4NF)

All multi-valued dependencies were already removed in 1NF. Therefore, the schema satisfies Fourth Normal Form.

Fifth Normal Form (5NF)

All relations are decomposed into lossless joins and no further decomposition is possible without loss of information. The schema satisfies Fifth Normal Form.

FINAL RELATIONS

➤ DEPARTMENT

DepartmentID, DepartmentName, ManagerID

➤ JOB_POSITION

JobPositionID, PositionTitle, PayGrade, JobDescription, BaseSalary

➤ EMPLOYEE_ID_MAPPING

CNIC, *EmployeeID*

➤ EMPLOYEE

EmployeeID, FirstName, LastName, HireDate, Email, *DepartmentID*, *JobPositionID*, ManagerID, EmploymentType

➤ EMPLOYEE_CONTACT

ContactID, *EmployeeID*, ContactType, ContactNumber

➤ ATTENDANCE

AttendanceID, *EmployeeID*, Date, CheckInTime, CheckOutTime, Status

➤ LEAVE_RECORDS

LeaveID, *EmployeeID*, LeaveType, StartDate, EndDate, Status

➤ OVERTIME_RECORDS

OvertimeID, *EmployeeID*, Date, HoursWorked, OvertimeRate

Employee Payroll Management System

➤ BENEFITS

BenefitID, EmployeeID, BenefitType, EffectiveDate

➤ INSURANCE

BenefitID, InsuranceProvider, CoverageAmount

➤ ALLOWANCE

BenefitID, AllowanceType, MonthlyAmount

➤ DEDUCTIONS

DeductionID, EmployeeID, DeductionName, Amount, EffectiveDate, Reason

➤ TAX_RECORDS

TaxID, EmployeeID, TaxYear, TaxAmount, TaxType

➤ PAYROLL

PayrollID, EmployeeID, PayrollMonth, PayrollYear, BasicSalary

➤ PAYROLL_DEDUCTIONS

PayrollID, DeductionID, Amount

(Composite Primary Key)

➤ PAYROLL_BENEFITS

PayrollID, BenefitID, Amount

(Composite Primary Key)

➤ SALARY_SLIP

SlipID, PayrollID, GeneratedDate, PDFFilePath

DESCRIPTION OF RELATIONS

1. EMPLOYEE

(Employee_ID, First_Name, Last_Name, CNIC, Contact_Number, HireDate, Email, Department_ID, Job_Position_ID, Manager_ID)

STRONG ENTITY

Attributes:

- **Employee_ID:** Unique id of every employee PK
- **First_Name:** Simple First name of employee
- **Last_Name:** Simple last name of employee
- **CNIC:** Unique cnic number of employee
- **Contact_Number:** Simple contact number of employee
- **HireDate:** Simple hiring date
- **Email:** Simple email addresses of employee

Employee Payroll Management System

- **Department_ID**: FK reference from DEPARTMENT
- **Job_Position_ID**: FK reference from JOB POSITION
- **Manager_ID**: FK

Primary Key (PK): Employee_ID

Foreign Key (FK): Manager_ID referenced Employee(ManagerID)

2. DEPARTMENT

(Department_ID, Department_Name, Manager_ID)

STRONG ENTITY

Attributes:

- **Department_ID**: PK uniquely identifies each employee
- **Department_Name**: Simple Name of employee
- **Manager_ID**: FK referenced from employee

Primary Key (PK): Department_ID

Foreign Key (FK): Manager_ID referenced Employee(ManagerID)

3. JOB POSITION

(Job_Position_ID, Position_Title, PayGrade, Description)

STRONG ENTITY

Attributes:

- **Job_Position_ID**: PK uniquely identifies each job
- **Position_Title**: Simple position title of each job
- **PayGrade**: Simple paygrade of employees
- **Description**: Simple job description

Primary Key (PK): Job_Position_ID

4. ATTENDANCE

(Attendance_ID, Employee_ID, Date, Check_In_Time, Check_Out_Time, Status
(Present/Absent/Late))

STRONG ENTITY

Attributes:

- **Attendance_ID**: PK uniquely identifies each employee's attendance
- **Employee_ID**: FK referenced from employee

Employee Payroll Management System

- **Date:** Simple date
- **Check_In_Time:** Simple time when employee check in
- **Check_Out_Time:** Simple time when employee leaves/checkout
- **Status (Present/Absent/Late):** Simple showing whether employee present absent or on leave

Primary Key (PK): Attendance_ID

Foreign Key (FK): Employee_ID referenced Employee(EmployeeID)

5. LEAVE RECORDS

(Leave_ID, Employee_ID, Leave_Type, Start_Date, End_Date, Status (Approved/Pending/Rejected))

STRONG ENTITY

Attributes:

- **Leave_ID:** PK unique id of each employee's each leave
- **Employee_ID:** FK referenced from employee
- **Leave_Type:** Simple type of leave
- **Start_Date:** Simple the starting date of leave
- **End_Date:** Simple end date of leave
- **Status (Approved/Pending/Rejected):** Simple status of leave

Primary Key (PK): Leave_ID

Foreign Key (FK): Employee_ID referenced Employee(EmployeeID)

6. OVERTIME RECORDS

(Overtime_ID, Employee_ID, Date, Hours_Worked, Overtime_Rate)

STRONG ENTITY

Attributes:

- **Overtime_ID:** PK uniquely identifies overtime of employee's
- **Employee_ID:** FK referenced from employee
- **Date:** Simple date
- **Hours_Worked:** Simple hours each employee worked
- **Overtime_Rate:** Simple overtime each employee gave

Primary Key (PK): Overtime_ID

Foreign Key (FK): Employee_ID referenced Employee(EmployeeID)

7. BENEFITS

(Benefit_ID, Employee_ID, Benefit_Type (Insurance/Allowance), Effective_Date)

SUPERTYPE

Attributes:

- **Benefit_ID**: PK uniquely identifies each employee's benefits
- **Employee_ID**: FK referenced from employee
- **Benefit_Type** (Insurance/Allowance): Discriminator type of benefit
- **Effective_Date**: Simple tells the effective date

Primary Key (PK): Benefit_ID

Foreign Key (FK): Employee_ID referenced Employee(EmployeeID)

8. INSURANCE

(BenefitID, InsuranceProvider, CoverageAmount)

SUBTYPE

Attributes:

- **Benefit_ID**: PK uniquely identifies employee's benefit FK referenced from parent class
- **Insurance_Provider**: Simple the provider of insurance
- **Coverage_Amount**: Simple total amount that covers

9. ALLOWANCE

(Benefit_ID, Allowance_Type, Monthly_Amount)

SUBTYPE

Attributes:

- **Benefit_ID**: PK uniquely identifies employee's benefit FK referenced from parent class
- **Allowance_Type**: Simple type of allowance
- **Monthly_Amount**: Simple monthly amount paid in allowance

10. DEDUCTIONS

(Deduction_ID, Employee_ID, Deduction_Name, Amount, Effective_Date, Reason)

STRONG ENTITY

Attributes:

- **Deduction_ID**: PK uniquely identifies each employee's deduction
- **Employee_ID**: FK referenced from employee
- **Deduction_Name**: Simple Type/name of deduction

Employee Payroll Management System

- **Amount:** Simple amount for deduction
- **Effective_Date:** Simple dates in deduction
- **Reason:** Simple reason for date

Primary Key (PK): Deduction_ID

Foreign Key (FK): Employee_ID referenced Employee(EmployeeID)

11. PAYROLL

(Payroll_ID, Employee_ID, Payroll_Month, Payroll_Year, Basic_Salary, Total_Deductions, Total_Benefits, Net_Salary)

STRONG ENTITY

Attributes:

- **Payroll_ID:** PK uniquely identifies each employee's
- **Employee_ID:** FK referenced from employee
- **Payroll_Month:** Simple month of payroll
- **Payroll_Year:** Simple year of payroll
- **Basic_Salary:** Simple basic salary
- **Total_Deductions:** Simple total deductions from salary
- **Total_Benefits:** Simple total benefits granted
- **Net_Salary:** Simple net salary of employee

Primary Key (PK): Payroll_ID

Foreign Key (FK): Employee_ID referenced Employee(EmployeeID)

12. SALARY SLIP

(Slip_ID, Payroll_ID, Generated_Date, PDF_File_Path)

Attributes:

- **Slip_ID:** PK uniquely identifies each employee slip
- **Payroll_ID:** FK referenced from payroll
- **Generated_Date:** Simple date of generation of payroll
- **PDF_File_Path:** Simple pdf file path

Primary Key (PK): Slip_ID

Foreign Key (FK): Payroll_ID referenced Payroll(PayrollID)

13. TAX RECORDS

(Tax_ID, Employee_ID, Tax_Year, Tax_Amount, Tax_Type)

Attributes:

Employee Payroll Management System

- **Tax_ID:** PK uniquely identifies each employee's tax
- **Employee_ID:** FK referenced from employee
- **Tax_Year:** Simple full year tax
- **Tax_Amount:** Simple amount of tax
- **Tax_Type:** Simple type of tax

Primary Key (PK): Tax_ID

Foreign Key (FK): Employee_ID referenced EmployeeEmployeeID

DESCRIPTION OF RELATIONS(In table form)

EMPLOYEE

Attribute	Data Type	Size	Constraints
Employee_ID	Number	4	Primary Key
First_Name	Varchar2	50	Not Null
Last_Name	Varchar2	50	Not Null
CNIC	Varchar2	50	Unique ,Check(13 digits)
Contact_Number	Varchar2	50	Not Null
Hiredate	Date	N\A	None
Email	Varchar2	50	Not Null
Department_ID	Number	4	Foreign Key references Dept Table
Job Position_ID	Number	4	Foreign Key references Job Position Table
Manager_ID	Number	4	Foreign Key references Employee Table(Referenced Employee - Manager)

DEPARTMENT

Attribute	Data Type	Size	Constraints
Department_ID	Number	4	Primary Key

Employee Payroll Management System

Department_Name	Varchar2	50	Not Null
Manager_ID	Number	4	Foreign Key references Employee Table(References Employee-Manager)

JOB POSITION

Attribute	Data Type	Size	Constraints
Job_Position_ID	Number	4	Primary Key
Position_Title	Varchar2	50	Not Null
PayGrade	Varchar2	50	
Description	Varchar2	50	Not Null

ATTENDANCE

Attribute	Data Type	Size	Constraints
Attendance_ID	Number	4	Primary Key
Employee_ID	Number	4	Foreign key references Employee Table(Employee_ID)
Date	Date	N\A	Not Null
Check_In_Time	Number	4	Not Null
Check_Out_Time	Number	4	Not Null
Status(Absent\Present\Leave)	Char(2)	N\A	Check(Status IN('P', 'L', 'A'))

LEAVE RECORDS

Attribute	Data Type	Size	Constraints
Leave_ID	Number	4	Primary Key
Employee_ID	Number	4	Foreign key references Employee Table(Employee_ID)
Leave_Type	Varchar2	50	Not Null
Start_Date	Date	N\A	Not Null

Employee Payroll Management System

End_Date	Date	N\A	Not Null
Status(Approved\Pending\Rejected)	Char(2)	N\A	Check(Status IN('A', 'P', 'R'))

OVERTIME RECORDS

Attribute	Data Type	Size	Constraints
OverTime_ID	Number	4	Primary Key
Employee_ID	Number	4	Foreign key references Employee Table(Employee_ID)
Date	Date	N\A	Not Null
Hours_Worked	Number	4	Not Null
OverTime_Rate	Number	4	Not Null

BENEFITS

Attribute	Data Type	Size	Constraints
Benefit_ID	Number	4	Primary Key
Employee_ID	Number	4	Foreign key references Employee Table(Employee_ID)
Benefit_Type(Insurance\Allowance)	Char(2)	N\A	Check(Benefit_Type IN('I', 'A'))
Effective_Date	Date	N\A	Not Null

INSURANCE

Attribute	Data Type	Size	Constraints
Benefit_ID	Number	4	Primary Key
Insurance_Provider	Varchar2	50	Not Null
Coverage_Amount	Number	4	Not Null

ALLOWANCE

Attribute	Data Type	Size	Constraints
-----------	-----------	------	-------------

Employee Payroll Management System

Benefit_ID	Number	4	Primary Key
Allowance_Type	Varchar2	50	Not Null
Monthly_Amount	Number	4	Not Null

DEDUCTIONS

Attribute	Data Type	Size	Constraints
Deduction_ID	Number	4	
Employee_ID	Number	4	Foreign key references Employee Table(Employee_ID)
Deduction_Name	Varchar2	50	Not Null
Amount	Number	4	Not Null
Effective_Date	Date	N\A	Not Null
Reason	Varchar2	50	Not Null

PAYROLL

Attribute	Data Type	Size	Constraints
Payroll_ID	Number	4	Primary Key
Employee_ID	Number	4	Foreign key references Employee Table(Employee_ID)
Payroll_Month	Number	4	Not Null
Payroll_Year	Number	4	Not Null
Basic_Salary	Number	4	Not Null
Total_Deductions	Number	4	None
Total_Benefits	Number	4	None
Net_Salary	Number	4	Not Null

SALARY SLIP

Attribute	Data Type	Size	Constraints
-----------	-----------	------	-------------

Employee Payroll Management System

Slip_ID	Number	4	Primary Key
Payroll_ID	Number	4	Foreign Key Refernces to PAYROLL(Payroll_ID)
Generated_Date	Date	N\A	Not Null
PDF_File_Path	Varchar2	50	None

TAX RECORDS

Attribute	Data Type	Size	Constraints
Tax_ID	Number	4	Primary Key
Employee_ID	Number	4	Foreign key references Employee Table(Employee_ID)
Tax_Year	Number	4	Not Null
Tax_Amount	Number	4	Not Null
Tax_Type	Varchar2	50	None

TABLES CREATION USING SQL

1) EMPLOYEE TABLE

```
CREATE TABLE EMPLOYEE (  
    Employee_ID    NUMBER(4) PRIMARY KEY,  
    First_Name     VARCHAR2(50) NOT NULL,  
    Last_Name      VARCHAR2(50) NOT NULL,  
    CNIC           VARCHAR2(50) UNIQUE  
                CHECK (LENGTH(CNIC) = 13),  
    Contact_Number VARCHAR2(50) NOT NULL,  
    HireDate       DATE,  
    Email          VARCHAR2(50) NOT NULL,  
    Department_ID  NUMBER(4),  
    Job_Position_ID NUMBER(4),  
    Manager_ID     NUMBER(4),  
  
    CONSTRAINT fk_emp_dept  
        FOREIGN KEY (Department_ID) REFERENCES DEPARTMENT(Department_ID),
```

Employee Payroll Management System

CONSTRAINT fk_emp_job

FOREIGN KEY (Job_Position_ID) REFERENCES JOB_POSITION(Job_Position_ID),

CONSTRAINT fk_emp_manager

FOREIGN KEY (Manager_ID) REFERENCES EMPLOYEE(Employee_ID)

);

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
EMPLOYEE	EMPLOYEE_ID	NUMBER	-	4	0	1	-	-	-
	FIRST_NAME	VARCHAR2	50	-	-	-	-	-	-
	LAST_NAME	VARCHAR2	50	-	-	-	-	-	-
	CNIC	VARCHAR2	50	-	-	-	✓	-	-
	CONTACT_NUMBER	VARCHAR2	50	-	-	-	-	-	-
	HIREDATE	DATE	7	-	-	-	✓	-	-
	EMAIL	VARCHAR2	50	-	-	-	-	-	-
	DEPARTMENT_ID	NUMBER	-	4	0	-	✓	-	-
	JOB_POSITION_ID	NUMBER	-	4	0	-	✓	-	-
	MANAGER_ID	NUMBER	-	4	0	-	✓	-	-
									1 - 10

2) DEPARTMENT TABLE

CREATE TABLE DEPARTMENT (

Department_ID NUMBER(4) PRIMARY KEY,

Department_Name VARCHAR2(50) NOT NULL,

Manager_ID NUMBER(4),

CONSTRAINT fk_dept_manager

FOREIGN KEY (Manager_ID) REFERENCES EMPLOYEE(Employee_ID)

);

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
DEPARTMENT	DEPARTMENT_ID	NUMBER	-	4	0	1	-	-	-
	DEPARTMENT_NAME	VARCHAR2	50	-	-	-	-	-	-
	MANAGER_ID	NUMBER	-	4	0	-	✓	-	-
									1 - 3

3) JOB_POSITION

CREATE TABLE JOB_POSITION (

Job_Position_ID NUMBER(4) PRIMARY KEY,

Position_Title VARCHAR2(50) NOT NULL,

PayGrade VARCHAR2(50),

Description VARCHAR2(50) NOT NULL

);

Employee Payroll Management System

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>JOB_POSITION</u>	<u>JOB_POSITION_ID</u>	NUMBER	-	4	0	1	-	-	-
	<u>POSITION_TITLE</u>	VARCHAR2	50	-	-	-	-	-	-
	<u>PAYGRADE</u>	VARCHAR2	50	-	-	-	✓	-	-
	<u>DESCRIPTION</u>	VARCHAR2	50	-	-	-	-	-	-
1 - 4									

4) ATTENDANCE

```

CREATE TABLE ATTENDANCE (
    Employee_ID    NUMBER(4),
    Date_Recorded  DATE,
    Check_In_Time  NUMBER(4) NOT NULL,
    Check_Out_Time NUMBER(4) NOT NULL,
    Status         CHAR(2)
    CHECK (Status IN ('P','A','L')),

    CONSTRAINT pk_attendance
        PRIMARY KEY (Employee_ID, Date_Recorded),

    CONSTRAINT fk_att_emp
        FOREIGN KEY (Employee_ID)
        REFERENCES EMPLOYEE(Employee_ID)
);

```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>ATTENDANCE</u>	<u>EMPLOYEE_ID</u>	NUMBER	-	4	0	1	-	-	-
	<u>DATE_RECORDED</u>	DATE	7	-	-	2	-	-	-
	<u>CHECK_IN_TIME</u>	NUMBER	-	4	0	-	-	-	-
	<u>CHECK_OUT_TIME</u>	NUMBER	-	4	0	-	-	-	-
	<u>STATUS</u>	CHAR	2	-	-	-	✓	-	-
1 - 5									

5) LEAVE_RECORDS

```

CREATE TABLE LEAVE_RECORDS (
    Leave_ID    NUMBER(4) PRIMARY KEY,
    Employee_ID NUMBER(4),
    Leave_Type  VARCHAR2(50) NOT NULL,
    Start_Date  DATE NOT NULL,
    End_Date    DATE NOT NULL,
    Status      CHAR(2)
    CHECK (Status IN ('A','P','R')),

    CONSTRAINT fk_leave_emp

```

Employee Payroll Management System

FOREIGN KEY (Employee_ID) REFERENCES EMPLOYEE(Employee_ID)
);

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
LEAVE_RECORDS	LEAVE_ID	NUMBER	-	4	0	1	-	-	-
	EMPLOYEE_ID	NUMBER	-	4	0	-	✓	-	-
	LEAVE_TYPE	VARCHAR2	50	-	-	-	-	-	-
	START_DATE	DATE	7	-	-	-	-	-	-
	END_DATE	DATE	7	-	-	-	-	-	-
	STATUS	CHAR	2	-	-	-	✓	-	-
1 - 6									

6) OVERTIME_RECORDS

CREATE TABLE OVERTIME_RECORDS (
OverTime_ID NUMBER(4) PRIMARY KEY,
Employee_ID NUMBER(4),
Date_Recorded DATE NOT NULL,
Hours_Worked NUMBER(4) NOT NULL,
OverTime_Rate NUMBER(4) NOT NULL,

CONSTRAINT fk_ot_emp

FOREIGN KEY (Employee_ID) REFERENCES EMPLOYEE(Employee_ID)
);

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
OVERTIME_RECORDS	OVERTIME_ID	NUMBER	-	4	0	1	-	-	-
	EMPLOYEE_ID	NUMBER	-	4	0	-	✓	-	-
	DATE_RECORDED	DATE	7	-	-	-	-	-	-
	HOURS_WORKED	NUMBER	-	4	0	-	-	-	-
	OVERTIME_RATE	NUMBER	-	4	0	-	-	-	-
1 - 5									

7) BENEFITS

CREATE TABLE BENEFITS (
Benefit_ID NUMBER(4) PRIMARY KEY,
Employee_ID NUMBER(4),
Benefit_Type CHAR(2)
CHECK (Benefit_Type IN ('T','A')),
Effective_Date DATE NOT NULL,

CONSTRAINT fk_benefit_emp

FOREIGN KEY (Employee_ID) REFERENCES EMPLOYEE(Employee_ID)
);

Employee Payroll Management System

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>BENEFITS</u>	<u>BENEFIT_ID</u>	NUMBER	-	4	0	1	-	-	-
	<u>EMPLOYEE_ID</u>	NUMBER	-	4	0	-	✓	-	-
	<u>BENEFIT_TYPE</u>	CHAR	2	-	-	-	✓	-	-
	<u>EFFECTIVE_DATE</u>	DATE	7	-	-	-	-	-	-
1 - 4									

8) INSURANCE

```
CREATE TABLE INSURANCE (
    Benefit_ID      NUMBER(4) PRIMARY KEY,
    Insurance_Provider VARCHAR2(50) NOT NULL,
    Coverage_Amount  NUMBER(4) NOT NULL,

    CONSTRAINT fk_ins_benefit
    FOREIGN KEY (Benefit_ID) REFERENCES BENEFITS(Benefit_ID)
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>INSURANCE</u>	<u>BENEFIT_ID</u>	NUMBER	-	4	0	1	-	-	-
	<u>INSURANCE_PROVIDER</u>	VARCHAR2	50	-	-	-	-	-	-
	<u>COVERAGE_AMOUNT</u>	NUMBER	-	4	0	-	-	-	-
1 - 3									

9) ALLOWANCE

```
CREATE TABLE ALLOWANCE (
    Benefit_ID      NUMBER(4) PRIMARY KEY,
    Allowance_Type  VARCHAR2(50) NOT NULL,
    Monthly_Amount  NUMBER(4) NOT NULL,

    CONSTRAINT fk_allow_benefit
    FOREIGN KEY (Benefit_ID) REFERENCES BENEFITS(Benefit_ID)
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>ALLOWANCE</u>	<u>BENEFIT_ID</u>	NUMBER	-	4	0	1	-	-	-
	<u>ALLOWANCE_TYPE</u>	VARCHAR2	50	-	-	-	-	-	-
	<u>MONTHLY_AMOUNT</u>	NUMBER	-	4	0	-	-	-	-
1 - 3									

10) DEDUCTIONS

```
CREATE TABLE DEDUCTIONS (
    Deduction_ID  NUMBER(4) PRIMARY KEY,
```

Employee Payroll Management System

Employee_ID NUMBER(4),
Deduction_Name VARCHAR2(50) NOT NULL,
Amount NUMBER(4) NOT NULL,
Effective_Date DATE NOT NULL,
Reason VARCHAR2(50) NOT NULL,

CONSTRAINT fk_deduct_emp

FOREIGN KEY (Employee_ID) REFERENCES EMPLOYEE(Employee_ID)

);

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>DEDUCTIONS</u>	<u>DEDUCTION_ID</u>	NUMBER	-	4	0	1	-	-	-
	<u>EMPLOYEE_ID</u>	NUMBER	-	4	0	-	✓	-	-
	<u>DEDUCTION_NAME</u>	VARCHAR2	50	-	-	-	-	-	-
	<u>AMOUNT</u>	NUMBER	-	4	0	-	-	-	-
	<u>EFFECTIVE_DATE</u>	DATE	7	-	-	-	-	-	-
	<u>REASON</u>	VARCHAR2	50	-	-	-	-	-	-
									1 - 6

11) PAYROLL

CREATE TABLE PAYROLL (

Payroll_ID NUMBER(4) PRIMARY KEY,
Employee_ID NUMBER(4),
Payroll_Month NUMBER(4) NOT NULL,
Payroll_Year NUMBER(4) NOT NULL,
Basic_Salary NUMBER(4) NOT NULL,
Total_Deductions NUMBER(4),
Total_Benefits NUMBER(4),
Net_Salary NUMBER(4) NOT NULL,

CONSTRAINT fk_payroll_emp

FOREIGN KEY (Employee_ID) REFERENCES EMPLOYEE(Employee_ID)

);

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>PAYROLL</u>	<u>PAYROLL_ID</u>	NUMBER	-	4	0	1	-	-	-
	<u>EMPLOYEE_ID</u>	NUMBER	-	4	0	-	✓	-	-
	<u>PAYROLL_MONTH</u>	NUMBER	-	4	0	-	-	-	-
	<u>PAYROLL_YEAR</u>	NUMBER	-	4	0	-	-	-	-
	<u>BASIC_SALARY</u>	NUMBER	-	4	0	-	-	-	-
	<u>TOTAL_DEDUCTIONS</u>	NUMBER	-	4	0	-	✓	-	-
	<u>TOTAL_BENEFITS</u>	NUMBER	-	4	0	-	✓	-	-
	<u>NET_SALARY</u>	NUMBER	-	4	0	-	-	-	-
									1 - 8

12) SALARY_SLIP

```
CREATE TABLE SALARY_SLIP (
    Slip_ID    NUMBER(4) PRIMARY KEY,
    Payroll_ID NUMBER(4),
    Generated_Date DATE NOT NULL,
    PDF_File_Path VARCHAR2(50),

    CONSTRAINT fk_slip_payroll
    FOREIGN KEY (Payroll_ID) REFERENCES PAYROLL(Payroll_ID)
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>SALARY_SLIP</u>	<u>SLIP_ID</u>	NUMBER	-	4	0	1	-	-	-
	<u>PAYROLL_ID</u>	NUMBER	-	4	0	-	✓	-	-
	<u>GENERATED_DATE</u>	DATE	7	-	-	-	-	-	-
	<u>PDF_FILE_PATH</u>	VARCHAR2	50	-	-	-	✓	-	-
1 - 4									

13) TAX_RECORDS

```
CREATE TABLE TAX_RECORDS (
    Tax_ID    NUMBER(4) PRIMARY KEY,
    Employee_ID NUMBER(4),
    Tax_Year   NUMBER(4) NOT NULL,
    Tax_Amount NUMBER(4) NOT NULL,
    Tax_Type   VARCHAR2(50),

    CONSTRAINT fk_tax_emp
    FOREIGN KEY (Employee_ID) REFERENCES EMPLOYEE(Employee_ID)
);
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>TAX_RECORDS</u>	<u>TAX_ID</u>	NUMBER	-	4	0	1	-	-	-
	<u>EMPLOYEE_ID</u>	NUMBER	-	4	0	-	✓	-	-
	<u>TAX_YEAR</u>	NUMBER	-	4	0	-	-	-	-
	<u>TAX_AMOUNT</u>	NUMBER	-	4	0	-	-	-	-
	<u>TAX_TYPE</u>	VARCHAR2	50	-	-	-	✓	-	-
1 - 5									

INSERTION OF DATA IN THE TABLES**1) EMPLOYEE**

Employee Payroll Management System

- INSERT INTO EMPLOYEE (Employee_ID, First_Name, Last_Name, CNIC, Contact_Number, HireDate, Email, Department_ID, Job_Position_ID, Manager_ID) VALUES (1001, 'Abriza', 'Amin', '3520123456789', '03001234567', TO_DATE('2022-01-10','YYYY-MM-DD'), 'abriza.amin@email.com', 101, 201, NULL);
- INSERT INTO EMPLOYEE (Employee_ID, First_Name, Last_Name, CNIC, Contact_Number, HireDate, Email, Department_ID, Job_Position_ID, Manager_ID) VALUES (1002, 'Fiza', 'Majeed', '3520123456790', '03007654321', TO_DATE('2022-03-15','YYYY-MM-DD'), 'fiza.majeed@email.com', 102, 202, 1001);
- INSERT INTO EMPLOYEE (Employee_ID, First_Name, Last_Name, CNIC, Contact_Number, HireDate, Email, Department_ID, Job_Position_ID, Manager_ID) VALUES (1003, 'Mahnoor', 'Ghani', '3520123456791', '03009876543', TO_DATE('2023-02-20','YYYY-MM-DD'), 'mahnoor.ghani@email.com', 101, 203, 1001);

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	CNIC	CONTACT_NUMBER
1001	Abriza	Amin	3520123456789	03001234567
1002	Fiza	Majeed	3520123456790	03007654321
1003	Mahnoor	Ghani	3520123456791	03009876543

HIREDATE	EMAIL	DEPARTMENT_ID	JOB_POSITION_ID	MANAGER_ID
01/10/2022	abriza.amin@email.com	101	201	-
03/15/2022	fiza.majeed@email.com	102	202	1001
02/20/2023	mahnoor.ghani@email.com	101	203	1001

2) DEPARTMENT

- INSERT INTO DEPARTMENT (Department_ID, Department_Name, Manager_ID) VALUES (101, 'HR', 1001);
- INSERT INTO DEPARTMENT (Department_ID, Department_Name, Manager_ID) VALUES (102, 'Finance', 1002);

DEPARTMENT_ID	DEPARTMENT_NAME	MANAGER_ID
101	HR	1001
102	Finance	1002

3) JOB POSITION

- INSERT INTO JOB_POSITION (Job_Position_ID, Position_Title, PayGrade, Description)
VALUES (201, 'HR Manager', 'G5', 'Manages HR activities');
- INSERT INTO JOB_POSITION (Job_Position_ID, Position_Title, PayGrade, Description)
VALUES (202, 'Accountant', 'G4', 'Handles accounting and payroll');
- INSERT INTO JOB_POSITION (Job_Position_ID, Position_Title, PayGrade, Description)
VALUES (203, 'HR Executive', 'G3', 'Assists HR Manager');

JOB_POSITION_ID	POSITION_TITLE	PAYGRADE	DESCRIPTION
203	HR Executive	G3	Assists HR Manager
201	HR Manager	G5	Manages HR activities
202	Accountant	G4	Handles accounting and payroll

4) ATTENDANCE

- INSERT INTO ATTENDANCE (Employee_ID, Date_Recorded, Check_In_Time, Check_Out_Time, Status)
VALUES (1001, TO_DATE('2025-12-01','YYYY-MM-DD'), 900, 1700, 'P');
- INSERT INTO ATTENDANCE (Employee_ID, Date_Recorded, Check_In_Time, Check_Out_Time, Status)
VALUES (1002, TO_DATE('2025-12-01','YYYY-MM-DD'), 915, 1730, 'P');
- INSERT INTO ATTENDANCE (Employee_ID, Date_Recorded, Check_In_Time, Check_Out_Time, Status)
VALUES (1003, TO_DATE('2025-12-01','YYYY-MM-DD'), 930, 1700, 'L');

EMPLOYEE_ID	DATE_RECORDED	CHECK_IN_TIME	CHECK_OUT_TIME	STATUS
1001	12/01/2025	900	1700	P
1002	12/01/2025	915	1730	P
1003	12/01/2025	930	1700	L

5) LEAVE_RECORDS

- INSERT INTO LEAVE_RECORDS (Leave_ID, Employee_ID, Leave_Type, Start_Date, End_Date, Status)
VALUES (1, 1003, 'Sick Leave', TO_DATE('2025-12-05','YYYY-MM-DD'),
TO_DATE('2025-12-07','YYYY-MM-DD'), 'A');
- INSERT INTO LEAVE_RECORDS (Leave_ID, Employee_ID, Leave_Type, Start_Date, End_Date, Status)
VALUES (2, 1002, 'Casual Leave', TO_DATE('2025-12-10','YYYY-MM-DD'),
TO_DATE('2025-12-12','YYYY-MM-DD'), 'P');

LEAVE_ID	EMPLOYEE_ID	LEAVE_TYPE	START_DATE	END_DATE	STATUS
1	1003	Sick Leave	12/05/2025	12/07/2025	A
2	1002	Casual Leave	12/10/2025	12/12/2025	P

6) OVERTIME_RECORDS

- INSERT INTO OVERTIME_RECORDS (OverTime_ID, Employee_ID, Date_Recorded, Hours_Worked, OverTime_Rate)
VALUES (1, 1002, TO_DATE('2025-12-15','YYYY-MM-DD'), 5, 200);
- INSERT INTO OVERTIME_RECORDS (OverTime_ID, Employee_ID, Date_Recorded, Hours_Worked, OverTime_Rate)
VALUES (2, 1003, TO_DATE('2025-12-16','YYYY-MM-DD'), 3, 150);

OVERTIME_ID	EMPLOYEE_ID	DATE_RECORDED	HOURS_WORKED	OVERTIME_RATE
1	1002	12/15/2025	5	200
2	1003	12/16/2025	3	150

7) BENEFITS

- INSERT INTO BENEFITS (Benefit_ID, Employee_ID, Benefit_Type, Effective_Date)
VALUES (1, 1001, 'I', TO_DATE('2025-01-01','YYYY-MM-DD'));
- INSERT INTO BENEFITS (Benefit_ID, Employee_ID, Benefit_Type, Effective_Date)
VALUES (2, 1002, 'A', TO_DATE('2025-01-01','YYYY-MM-DD'));

Employee Payroll Management System

BENEFIT_ID	EMPLOYEE_ID	BENEFIT_TYPE	EFFECTIVE_DATE
1	1001	I	01/01/2025
2	1002	A	01/01/2025

8) INSURANCE

- INSERT INTO INSURANCE (Benefit_ID, Insurance_Provider, Coverage_Amount)
VALUES (1, 'State Life', 9999);

BENEFIT_ID	INSURANCE_PROVIDER	COVERAGE_AMOUNT
1	State Life	9999

9) ALLOWANCE

- INSERT INTO ALLOWANCE (Benefit_ID, Allowance_Type, Monthly_Amount)
VALUES (2, 'Transport', 5000);

BENEFIT_ID	ALLOWANCE_TYPE	MONTHLY_AMOUNT
2	Transport	5000

10) DEDUCTIONS

- INSERT INTO DEDUCTIONS (Deduction_ID, Employee_ID, Deduction_Name, Amount, Effective_Date, Reason)
VALUES (1, 1002, 'Late Penalty', 200, TO_DATE('2025-12-01','YYYY-MM-DD'), 'Late arrival');
- INSERT INTO DEDUCTIONS (Deduction_ID, Employee_ID, Deduction_Name, Amount, Effective_Date, Reason)
VALUES (2, 1003, 'Sick Leave Deduction', 300, TO_DATE('2025-12-05','YYYY-MM-DD'), 'Sick leave');

DEDUCTION_ID	EMPLOYEE_ID	DEDUCTION_NAME	AMOUNT	EFFECTIVE_DATE	REASON
1	1002	Late Penalty	200	12/01/2025	Late arrival
2	1003	Sick Leave Deduction	300	12/05/2025	Sick leave

11) PAYROLL

Employee Payroll Management System

- INSERT INTO PAYROLL (Payroll_ID, Employee_ID, Payroll_Month, Payroll_Year, Basic_Salary, Total_Deductions, Total_Benefits, Net_Salary)
VALUES (1, 1001, 12, 2025, 5000, 0, 500, 5500);
- INSERT INTO PAYROLL (Payroll_ID, Employee_ID, Payroll_Month, Payroll_Year, Basic_Salary, Total_Deductions, Total_Benefits, Net_Salary)
VALUES (2, 1002, 12, 2025, 4000, 200, 500, 4300);
- INSERT INTO PAYROLL (Payroll_ID, Employee_ID, Payroll_Month, Payroll_Year, Basic_Salary, Total_Deductions, Total_Benefits, Net_Salary)
VALUES (3, 1003, 12, 2025, 3500, 300, 0, 3200);

PAYROLL_ID	EMPLOYEE_ID	PAYROLL_MONTH	PAYROLL_YEAR	BASIC_SALARY	TOTAL_DEDUCTIONS	TOTAL_BENEFITS	NET_SALARY
1	1001	12	2025	5000	0	500	5500
2	1002	12	2025	4000	200	500	4300
3	1003	12	2025	3500	300	0	3200

12) SALARY_SLIP

- INSERT INTO SALARY_SLIP (Slip_ID, Payroll_ID, Generated_Date, PDF_File_Path)
VALUES (1, 1, TO_DATE('2025-12-31','YYYY-MM-DD'), '/slips/1001_Dec2025.pdf');
- INSERT INTO SALARY_SLIP (Slip_ID, Payroll_ID, Generated_Date, PDF_File_Path)
VALUES (2, 2, TO_DATE('2025-12-31','YYYY-MM-DD'), '/slips/1002_Dec2025.pdf');
- INSERT INTO SALARY_SLIP (Slip_ID, Payroll_ID, Generated_Date, PDF_File_Path)
VALUES (3, 3, TO_DATE('2025-12-31','YYYY-MM-DD'), '/slips/1003_Dec2025.pdf');

SLIP_ID	PAYROLL_ID	GENERATED_DATE	PDF_FILE_PATH
1	1	12/31/2025	/slips/1001_Dec2025.pdf
2	2	12/31/2025	/slips/1002_Dec2025.pdf
3	3	12/31/2025	/slips/1003_Dec2025.pdf

13) TAX_RECORDS

- INSERT INTO TAX_RECORDS (Tax_ID, Employee_ID, Tax_Year, Tax_Amount, Tax_Type)
VALUES (1, 1001, 2025, 5000, 'Income Tax');
- INSERT INTO TAX_RECORDS (Tax_ID, Employee_ID, Tax_Year, Tax_Amount, Tax_Type)
VALUES (2, 1002, 2025, 4000, 'Income Tax');

Employee Payroll Management System

- INSERT INTO TAX_RECORDS (Tax_ID, Employee_ID, Tax_Year, Tax_Amount, Tax_Type)
VALUES (3, 1003, 2025, 3500, 'Income Tax');

TAX_ID	EMPLOYEE_ID	TAX_YEAR	TAX_AMOUNT	TAX_TYPE
1	1001	2025	5000	Income Tax
2	1002	2025	4000	Income Tax
3	1003	2025	3500	Income Tax

VIEWS CREATION USING SQL

1) EMPLOYEE DETAILS VIEW

This view shows basic employee information along with their department and job position. It helps HR quickly access employee details without joining multiple tables.

```
CREATE VIEW VW_EMPLOYEE_DETAILS AS
SELECT
    E.Employee_ID,
    E.First_Name,
    E.Last_Name,
    E.Email,
    D.Department_Name,
    J.Position_Title
FROM EMPLOYEE E
JOIN DEPARTMENT D
    ON E.Department_ID = D.Department_ID
JOIN JOB_POSITION J
    ON E.Job_Position_ID = J.Job_Position_ID;
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>VW_EMPLOYEE_DETAILS</u>	<u>EMPLOYEE_ID</u>	NUMBER	-	4	0	-	-	-	-
	<u>FIRST_NAME</u>	VARCHAR2	50	-	-	-	-	-	-
	<u>LAST_NAME</u>	VARCHAR2	50	-	-	-	-	-	-
	<u>EMAIL</u>	VARCHAR2	50	-	-	-	-	-	-
	<u>DEPARTMENT_NAME</u>	VARCHAR2	50	-	-	-	-	-	-
	<u>POSITION_TITLE</u>	VARCHAR2	50	-	-	-	-	-	-
1 - 6									

2) PAYROLL SUMMARY VIEW

Employee Payroll Management System

This view displays salary information including basic salary, benefits, deductions, and net salary.

It is used by payroll staff to review and generate salary reports.

```
CREATE VIEW VW_PAYROLL_SUMMARY AS
SELECT
    E.Employee_ID,
    E.First_Name,
    E.Last_Name,
    P.Payroll_Month,
    P.Payroll_Year,
    P.Basic_Salary,
    P.Total_Benefits,
    P.Total_Deductions,
    P.Net_Salary
FROM EMPLOYEE E
JOIN PAYROLL P
ON E.Employee_ID = P.Employee_ID;
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
VW_PAYROLL_SUMMARY	EMPLOYEE_ID	NUMBER	-	4	0	-	-	-	-
	FIRST_NAME	VARCHAR2	50	-	-	-	-	-	-
	LAST_NAME	VARCHAR2	50	-	-	-	-	-	-
	PAYROLL_MONTH	NUMBER	-	4	0	-	-	-	-
	PAYROLL_YEAR	NUMBER	-	4	0	-	-	-	-
	BASIC_SALARY	NUMBER	-	4	0	-	-	-	-
	TOTAL_BENEFITS	NUMBER	-	4	0	-	✓	-	-
	TOTAL_DEDUCTIONS	NUMBER	-	4	0	-	✓	-	-
	NET_SALARY	NUMBER	-	4	0	-	-	-	-
1-9									

3) ATTENDANCE REPORT VIEW

This view shows daily attendance records of employees with check-in and check-out times.

It helps management monitor attendance and supports payroll calculations.

```
CREATE VIEW VW_ATTENDANCE_REPORT AS
SELECT
    E.Employee_ID,
    E.First_Name,
    E.Last_Name,
    A.Date_Recorded,
    A.Status,
    A.Check_In_Time,
    A.Check_Out_Time
```

Employee Payroll Management System

```
FROM EMPLOYEE E
JOIN ATTENDANCE A
ON E.Employee_ID = A.Employee_ID;
```

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
VW_ATTENDANCE_REPORT	EMPLOYEE_ID	NUMBER	-	4	0	-	-	-	-
	FIRST_NAME	VARCHAR2	50	-	-	-	-	-	-
	LAST_NAME	VARCHAR2	50	-	-	-	-	-	-
	DATE_RECORDED	DATE	7	-	-	-	-	-	-
	STATUS	CHAR	2	-	-	-	✓	-	-
	CHECK_IN_TIME	NUMBER	-	4	0	-	-	-	-
	CHECK_OUT_TIME	NUMBER	-	4	0	-	-	-	-
1-7									

SELECT STATEMENTS FOR COMMON REPORTS

1) EMPLOYEE PAYROLL SUMMARY REPORT

Shows employee info with net salary, total benefits, and deductions for a specific month/year.

```
SELECT
    e.Employee_ID,
    e.First_Name || ' ' || e.Last_Name AS Employee_Name,
    d.Department_Name,
    j.Position_Title,
    p.Payroll_Month,
    p.Payroll_Year,
    p.Basic_Salary,
    p.Total_Benefits,
    p.Total_Deductions,
    p.Net_Salary
FROM EMPLOYEE e
JOIN DEPARTMENT d ON e.Department_ID = d.Department_ID
JOIN JOB_POSITION j ON e.Job_Position_ID = j.Job_Position_ID
JOIN PAYROLL p ON e.Employee_ID = p.Employee_ID
WHERE p.Payroll_Month = 12
    AND p.Payroll_Year = 2025
ORDER BY p.Net_Salary DESC;
```


Employee Payroll Management System

EMPLOYEE_ID	EMPLOYEE_NAME	DEPARTMENT_NAME	POSITION_TITLE	PAYROLL_MONTH
1001	Abriza Amin	HR	HR Manager	12
1002	Fiza Majeed	Finance	Accountant	12
1003	Mahnoor Ghani	HR	HR Executive	12

PAYROLL_YEAR	BASIC_SALARY	TOTAL_BENEFITS	TOTAL_DEDUCTIONS	NET_SALARY
2025	5000	500	0	5500
2025	4000	500	200	4480
2025	3500	0	300	3470

2) DEPARTMENT-WISE SALARY EXPENSE REPORT

Helps management analyze salary expenses department-wise.

```
SELECT
    d.Department_Name,
    COUNT(e.Employee_ID) AS Total_Employees,
    SUM(p.Net_Salary) AS Total_Department_Salary
FROM EMPLOYEE e
JOIN DEPARTMENT d ON e.Department_ID = d.Department_ID
JOIN PAYROLL p ON e.Employee_ID = p.Employee_ID
GROUP BY d.Department_Name
ORDER BY Total_Department_Salary DESC;
```

DEPARTMENT_NAME	TOTAL_EMPLOYEES	TOTAL_DEPARTMENT_SALARY
HR	2	8700
Finance	1	4300

3) EMPLOYEES WITH OVERTIME AND TOTAL OVERTIME PAY

Shows employees who earned extra pay through overtime.

```
SELECT
    e.Employee_ID,
    e.First_Name || ' ' || e.Last_Name AS Employee_Name,
    SUM(o.Hours_Worked) AS Total_Overtime_Hours,
    SUM(o.Hours_Worked * o.OverTime_Rate) AS Total_Overtime_Pay
FROM EMPLOYEE e
JOIN OVERTIME_RECORDS o ON e.Employee_ID = o.Employee_ID
GROUP BY e.Employee_ID, e.First_Name, e.Last_Name
HAVING SUM(o.Hours_Worked) > 0;
```


Employee Payroll Management System

EMPLOYEE_ID	EMPLOYEE_NAME	TOTAL_OVERTIME_HOURS	TOTAL_OVERTIME_PAY
1002	Fiza Majeed	3	1200
1001	Abriza Amin	4	2000

4) EMPLOYEES EARNING ABOVE AVERAGE SALARY

Identifies high-earning employees compared to the company average.

```
SELECT
    e.Employee_ID,
    e.First_Name || ' ' || e.Last_Name AS Employee_Name,
    p.Net_Salary
FROM EMPLOYEE e
JOIN PAYROLL p ON e.Employee_ID = p.Employee_ID
WHERE p.Net_Salary >
    (SELECT AVG(Net_Salary) FROM PAYROLL);
```

EMPLOYEE_ID	EMPLOYEE_NAME	NET_SALARY
1001	Abriza Amin	5500

5) EMPLOYEES ON APROVED LEAVE

Displays all employees who are on approved leave.

```
SELECT
    e.Employee_ID,
    e.First_Name || ' ' || e.Last_Name AS Employee_Name,
    l.Leave_Type,
    l.Start_Date,
    l.End_Date
FROM EMPLOYEE e
JOIN LEAVE_RECORDS l ON e.Employee_ID = l.Employee_ID
WHERE l.Status = 'A'
ORDER BY l.Start_Date;
```

EMPLOYEE_ID	EMPLOYEE_NAME	LEAVE_TYPE	START_DATE	END_DATE
1003	Mahnoor Ghani	Casual	12/20/2025	12/21/2025

6) TAX DEDUCTION REPORT (WITH SALARY COMPARISON)

Shows tax deduction in comparison to employee salary.

```
SELECT
    e.Employee_ID,
    e.First_Name || ' ' || e.Last_Name AS Employee_Name,
    t.Tax_Year,
    t.Tax_Amount,
    p.Net_Salary
FROM EMPLOYEE e
JOIN TAX_RECORDS t ON e.Employee_ID = t.Employee_ID
JOIN PAYROLL p ON e.Employee_ID = p.Employee_ID
WHERE t.Tax_Year = p.Payroll_Year;
```

EMPLOYEE_ID	EMPLOYEE_NAME	TAX_YEAR	TAX_AMOUNT	NET_SALARY
1001	Abriza Amin	2025	5000	5500
1002	Fiza Majeed	2025	4000	4480
1003	Mahnoor Ghani	2025	3500	3470

FUNCTIONS

1) CALCULATE NET SALARY

Calculates net salary from basic salary, total benefits, and total deductions.

```
CREATE OR REPLACE FUNCTION calc_net_salary(
    p_basic_salary    NUMBER,
    p_total_benefits  NUMBER,
    p_total_deductions NUMBER
) RETURN NUMBER
IS
BEGIN
    RETURN p_basic_salary + NVL(p_total_benefits, 0) - NVL(p_total_deductions, 0);
END;
```

2) CALCULATE TOTAL OVERTIME PAY

Calculates total overtime payment for a specific employee.

```
CREATE OR REPLACE FUNCTION get_total_overtime(
```

Employee Payroll Management System

```
p_emp_id NUMBER
) RETURN NUMBER
IS
    v_total NUMBER;
BEGIN
    SELECT NVL(SUM(Hours_Worked * OverTime_Rate), 0)
    INTO v_total
    FROM OVERTIME_RECORDS
    WHERE Employee_ID = p_emp_id;

    RETURN v_total;
END;
```

STORED PROCEDURES

1) GENERATE PAYROLL RECORD

Automatically inserts payroll data for an employee, using net salary calculation.

```
CREATE OR REPLACE PROCEDURE generate_payroll(
    p_payroll_id NUMBER,
    p_emp_id     NUMBER,
    p_month      NUMBER,
    p_year       NUMBER,
    p_basic_salary NUMBER
)
IS
    v_benefits NUMBER;
    v_deductions NUMBER;
    v_net_salary NUMBER;
BEGIN
    SELECT NVL(SUM(Monthly_Amount), 0)
    INTO v_benefits
    FROM ALLOWANCE a
    JOIN BENEFITS b ON a.Benefit_ID = b.Benefit_ID
    WHERE b.Employee_ID = p_emp_id;

    SELECT NVL(SUM(Amount), 0)
    INTO v_deductions
    FROM DEDUCTIONS
    WHERE Employee_ID = p_emp_id;
```

Employee Payroll Management System

```
v_net_salary := calc_net_salary(p_basic_salary, v_benefits, v_deductions);
```

```
INSERT INTO PAYROLL(Payroll_ID, Employee_ID, Payroll_Month, Payroll_Year,
Basic_Salary, Total_Deductions, Total_Benefits, Net_Salary)
VALUES (p_payroll_id, p_emp_id, p_month, p_year, p_basic_salary, v_deductions,
v_benefits, v_net_salary);
END;
```

2) GENERATE SALARY SLIP

Creates a salary slip for payroll record.

```
CREATE OR REPLACE PROCEDURE generate_salary_slip(
    p_slip_id NUMBER,
    p_payroll_id NUMBER
)
IS
BEGIN
    INSERT INTO SALARY_SLIP(Slip_ID, Payroll_ID, Generated_Date, PDF_File_Path)
    VALUES (p_slip_id, p_payroll_id, SYSDATE, '/slips/generated.pdf');
END;
```

DATABASE TRIGGERS

1) PREVENT NEGATIVE NET SALARY

Prevents inserting or updating a payroll record with negative salary.

```
CREATE OR REPLACE TRIGGER trg_check_salary
BEFORE INSERT OR UPDATE ON PAYROLL
FOR EACH ROW
BEGIN
    IF :NEW.Net_Salary < 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Net salary cannot be negative');
    END IF;
END;
```

2) AUTO ATTENDANCE STATUS

Automatically marks employees **Late** if check-in is after 9 AM.

```
CREATE OR REPLACE TRIGGER trg_attendance_status
BEFORE INSERT ON ATTENDANCE
FOR EACH ROW
```

Employee Payroll Management System

```
BEGIN
  IF :NEW.Check_In_Time > 900 THEN
    :NEW.Status := 'L';
  ELSE
    :NEW.Status := 'P';
  END IF;
END;
```

SEQUENCES

1) EMPLOYEE_ID SEQUENCE

Automatically generate Employee_ID when inserting new employees.

```
CREATE SEQUENCE seq_employee_id
START WITH 1001
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

2) PAYROLL_ID SEQUENCE

For unique payroll records

```
CREATE SEQUENCE seq_payroll_id
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

3) LEAVE_ID SEQUENCE

Auto generate leave record ids.

```
CREATE SEQUENCE seq_leave_id
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;
```

4) OVERTIME_ID SEQUENCE

For unique overtime records.

```
CREATE SEQUENCE seq_overtime_id
START WITH 1
```

Employee Payroll Management System

INCREMENT BY 1
NOCACHE
NOCYCLE;

5) BENEFIT_ID SEQUENCE

CREATE SEQUENCE seq_benefit_id
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;

6) DEDUCTION_ID SEQUENCE

CREATE SEQUENCE seq_deduction_id
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;

7) TAX_ID SEQUENCE

CREATE SEQUENCE seq_tax_id
START WITH 1
INCREMENT BY 1
NOCACHE
NOCYCLE;

INDEXES

1) Index on EMPLOYEE Department

CREATE INDEX idx_emp_dept ON EMPLOYEE(Department_ID);

2) Index on EMPLOYEE Job_Position

CREATE INDEX idx_emp_job ON EMPLOYEE(Job_Position_ID);

3) Index on PAYROLL Employee_ID

CREATE INDEX idx_payroll_emp ON PAYROLL(Employee_ID);

4) Index on TAX_RECORDS Employee_ID

CREATE INDEX idx_tax_emp ON TAX_RECORDS(Employee_ID);

5) Index on ATTENDANCE Employee_ID

```
CREATE INDEX idx_attendance_emp ON ATTENDANCE(Employee_ID);
```

6) Index on LEAVE_RECORDS Employee_ID

```
CREATE INDEX idx_leave_emp ON LEAVE_RECORDS(Employee_ID);
```

7) Index on OVERTIME_RECORDS Employee_ID

```
CREATE INDEX idx_overtime_emp ON OVERTIME_RECORDS(Employee_ID);
```